Flexible Obstacle Interaction in Robotic Harvesting

Environment implementation report | ME5418 Machine Learning in Robotics | Group 18

1 Project overview

Our fully observed, two-dimensional Gymnasium environment models a circular robot end effector reaching fruit among flexible branches. Four branches form two opposing tip gaps. Each resting gap is narrower than the robot diameter, and its height varies between episodes. A policy can locate and gently open these internal passages or take a longer collision-free route around fixed anchors. Seeded geometry and bounded execution noise are configured separately.

2 Code explanation and organization

Component	Implementation
Environment	HarvestingEnv implements Gymnasium reset, step, render, and close. It maintains episode state, applies one of eight planar actions, computes rewards, and returns termination diagnostics.
Scene and physics	scene_generator.py creates two tip gates with alternating boundary and interior anchors. branch_physics.py projects normal contact and segment-length constraints. Anchors stay fixed; stiffness limits node movement, and springs restore the resting shape after release.
Observation and action	The 125-dimensional default observation contains robot/target positions, five padded branch slots with current/resting nodes, stiffness, contact, safe limits, masks, and remaining budget. Eight actions move in four directions by 0.025 or 0.060.
Reward and episode end	Reward combines progress, travel and deformation costs, a damage penalty, and a safe-success bonus. Peak deformation is checked inside every substep. Success or damage terminates an episode; 100 actions cause truncation. A* and the demo controller use identical low-level actions.

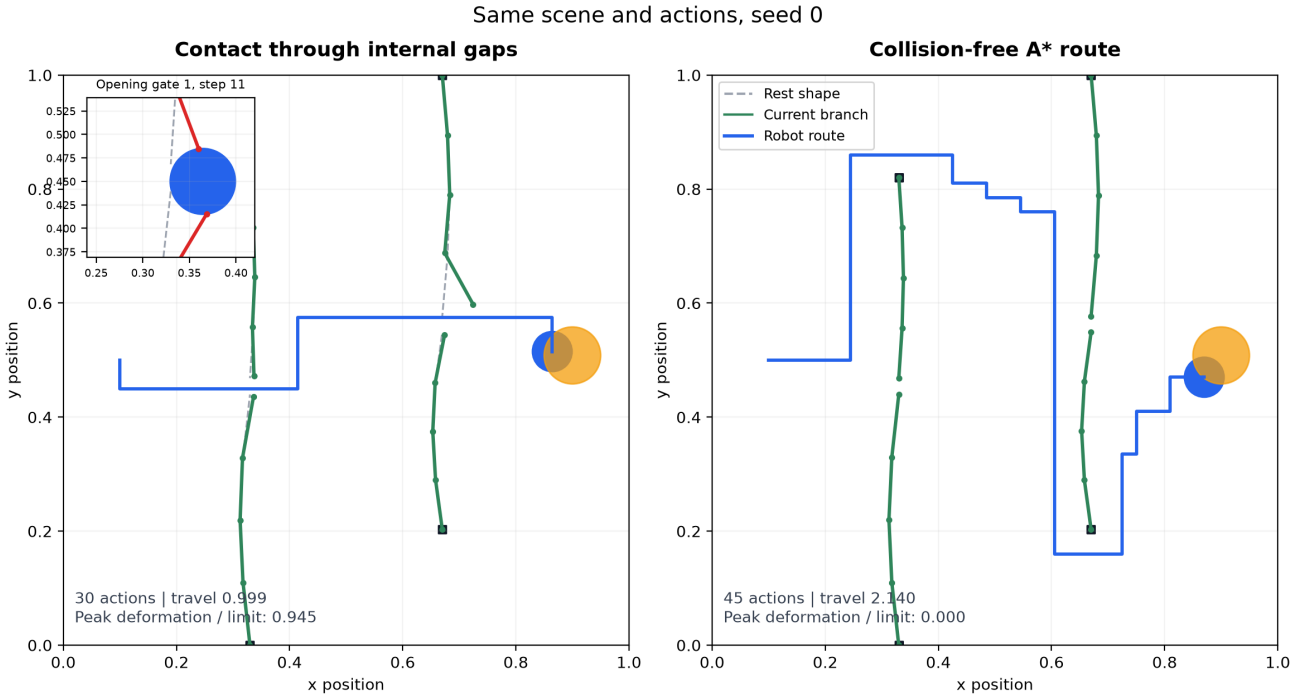


Figure 1. Same scene, seed 0. Left: the contact controller follows the internal gaps; the inset shows the first opening. Right: A* keeps branch geometry unchanged and travels around the alternating anchors.

3 Demonstration and reproducibility

The graphical demo saves GIFs, initial/contact/final snapshots, state traces, and JSON metrics. A zoom panel makes tip separation visible. On seed 0, the contact controller succeeded in 30 actions, travelled 0.999, and reached peak normalized deformation 0.945. A* succeeded in 45 actions with travel 2.140 and zero contact. Across seeds 0-9, both succeeded in all ten scenes; mean travel was 0.926 versus 2.177. These are environment diagnostics, not learned-policy evaluation.

Reproduction commands

Create environment	conda env create -f environment.yml
Activate	conda activate me5418-harvesting
Contact demo	python scripts/run_demo.py --policy push --seed 0
Same-scene detour	python scripts/run_demo.py --policy detour --seed 0
Run tests	pytest -q

4 Existing code, libraries, and tools

Task geometry, contact projections, rewards, A* search, and demonstration controllers are implemented in this repository. Gymnasium supplies the environment interface; NumPy handles geometry; Matplotlib and Pillow produce graphics; PyYAML loads configuration; pytest and Ruff support checks. No external robotics simulator is required. Codex assisted implementation, debugging, and documentation. Optional PyTorch code currently defines the DQN network; the training loop remains future work.

5 Reflections and lessons learned

Parallel branches with a shared top clearance made the original task an outer-detour problem. Opposing tips create a measurable contact opportunity: gaps of 0.028-0.043 are smaller than the robot diameter of 0.070. Random gap heights require alignment. Alternating interior anchors preserve a rigid-geometry route without a universal top or bottom shortcut. Motion is subdivided, difficult substeps are reduced, and contact/segment constraints are checked together to avoid tunnelling and numerical sticking.

Nine tests cover interface/reproducibility/rendering, gap geometry, rigid blocking, soft opening, fixed anchors, segment strain, and recovery. The geometric surrogate allows 3 percent segment strain and omits inertia, hysteresis, permanent deformation, and inter-branch mechanics. Its normalized parameters require physical calibration. One checked contact episode approaches its damage limit; ten successful seeds do not establish robustness. The contact controller is handwritten, and the default reward still prefers zero-contact detours despite their longer travel.

6 Remaining work toward the final project

Final work will implement DQN training and compare it with collision-free, myopic, and receding-horizon methods on held-out seeds. Gap width, stiffness, safety limits, reward weights, and execution noise need sensitivity tests. We will report success/damage, actions, travel, peak deformation, return, model evaluations, decision time, and training cost. Evidence of an RL advantage must come from that comparison, rather than from the scripted contact demonstration.